AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Michał Tokarz

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **4**

2.1. Teoretyczne podstawy i zastosowanie algorytmu listy dwukierunkowej . . 4

2.2. Sposób działania programu i mechanika pamięci 5

2.3. Ręczne śledzenie działania algorytmu (symulacja wstawiania) 5

2.4. Analiza wykorzystanych narzędzi deweloperskich 7

3. Projektowanie **9**

3.1. Środowisko pracy i wykorzystane technologie 9

3.2. Konfiguracja procesu budowania aplikacji 10

3.3. Architektura systemu – diagram klas (UML) 10

3.4. Zarządzanie kodem i automatyczna dokumentacja 11

4. Implementacja **13**

4.1. Paradygmat obiektowy i architektura plików 13

4.2. Realizacja wzorca projektowego Fabryka 13

4.3. Nawigacja po strukturze – wzorzec Iterator 14

4.4. Zarządzanie powiązaniami – operacje wstawiania 14

4.5. Zarządzanie pamięcią operacyjną i usuwanie elementów 16

4.6. Zestawienie wydajności operacji 17

4.7. Powstałe wyniki i zastosowanie narzędzi pomocniczych 18

5. Wnioski **19**

Literatura **20**

Spis rysunków **20**

Spis tabel **21**

Spis listingów **22**

1. Ogólne określenie wymagań

Celem niniejszej pracy jest zaprojektowanie oraz implementacja dwukierunkowej listy powiązanej w języku C++, operującej na stercie. Głównym założeniem projektowym jest praktyczne wykorzystanie paradygmatu programowania obiektowego oraz wzorców projektowych, w szczególności wzorca Fabryki (Factory) do zarządzania tworzeniem obiektów oraz wzorca Iteratora do bezpiecznej nawigacji po elementach kolekcji. Równie ważnym celem edukacyjnym jest zapoznanie się z systemem kontroli wersji Git (w tym obsługą platformy GitHub) oraz narzędziem Doxygen do tworzenia dokumentacji.

W ramach projektu chcemy uzyskać w pełni funkcjonalny program wieloplikowy, w którym każda klasa posiada wyodrębniony plik nagłówkowy i implementacyjny. Skupiamy się na zrealizowaniu podstawowych operacji na liście: dodawaniu elementów na początek, na koniec oraz pod wskazany indeks, a także na wyświetlaniu zawartości listy. System musi poprawnie zarządzać pamięcią (używając operatorów `new` i `delete`), aby zapobiec wyciekom.

Przewidywanym wynikiem pracy jest działająca i stabilna aplikacja konsolowa, poprawnie przechodząca testy w funkcji `main`. Ponadto, efektem prac będzie utworzone repozytorium Git zawierające odpowiednią historię commitów (dokumentującą etapy powstawania kodu) oraz automatycznie wygenerowana dokumentacja techniczna w formacie PDF przy użyciu oprogramowania Doxygen i środowiska LaTeX.

2. Analiza problemu

2.1. Teoretyczne podstawy i zastosowanie algorytmu listy dwukierunkowej

W inżynierii oprogramowania wybór odpowiedniej struktury danych jest kluczowy dla optymalizacji czasu wykonywania algorytmów oraz efektywnego zarządzania pamięcią operacyjną. Klasyczne, statyczne tablice (ang. *arrays*) charakteryzują się ciągłością w pamięci RAM, co pozwala na błyskawiczny dostęp do elementów za pomocą indeksów (złożoność $O(1)$). Posiadają one jednak poważne ograniczenia: ich rozmiar musi być z góry znany, a operacje wstawiania lub usuwania elementów w środku struktury wymagają przesuwania wszystkich kolejnych bloków danych, co drastycznie obniża wydajność (złożoność $O(n)$).

Odpowiedzią na te problemy jest lista dwukierunkowa (ang. *doubly linked list*). Jest to dynamiczna struktura danych, w której elementy (węzły) nie muszą być ułożone w pamięci obok siebie. Zamiast tego, każdy węzeł przechowuje informację o swoim położeniu względem innych poprzez wskaźniki (adresy w pamięci) kierujące do elementu poprzedniego i następnego. Dzięki temu rozmiar listy może rosnąć i maleć w czasie rzeczywistym, a wstawianie elementów w środek kolekcji – pod warunkiem znajomości dokładnego adresu docelowego – sprowadza się jedynie do szybkiej aktualizacji kilku wskaźników.

Z racji swojej elastyczności i możliwości swobodnego przemieszczania się w obu kierunkach, algorytmy oparte na liście dwukierunkowej znajdują powszechne zastosowanie w zaawansowanych systemach informatycznych. Główne obszary wykorzystania to między innymi:

- **Przeglądarki internetowe:** Historia odwiedzanych stron jest klasycznym przykładem listy dwukierunkowej. Użytkownik może płynnie przechodzić do strony poprzedniej (przycisk „Wstecz") oraz następnej (przycisk „Dalej"), a wywołanie nowego adresu ucina jedną ze ścieżek i nadpisuje wskaźniki.
- **Aplikacje biurowe i edytory graficzne:** Popularny mechanizm „Cofnij / Ponów" (Undo / Redo) opiera się na strukturach wskaźnikowych, gdzie każdy węzeł to jeden zapisany stan dokumentu.
- **Pamięć podręczna (Cache) i bazy danych:** Algorytmy takie jak LRU (Least Recently Used) wykorzystują listy dwukierunkowe połączone z tablicami mieszającymi (hash mapami) do błyskawicznego usuwania najdawniej używanych danych i wstawiania nowych na początek kolejki.

- **Odtwarzacze multimedialne:** Playlisty utworów muzycznych lub wideo pozwalają na zapętlanie (lista cykliczna) oraz nawigację do utworów poprzednich/następnych, co idealnie odwzorowuje strukturę węzłów i wskaźników.

2.2. Sposób działania programu i mechanika pamięci

Zaprojektowany program opiera się na dynamicznym zarządzaniu pamięcią (tzw. sterta – ang. *heap*). W przeciwieństwie do stosu (ang. *stack*), gdzie pamięć zwalniana jest automatycznie po wyjściu z bloku funkcji, sterta wymaga ręcznego przydzielania pamięci słowem kluczowym `new` oraz jej bezwzględnego zwalniania operatorem `delete`.

Podstawowym budulcem struktury jest klasa `Wezel`. Pełni ona funkcję kontenera przechowującego ładunek (w tym przypadku pojedynczą liczbę całkowitą `int`) oraz dwa wskaźniki sterujące. Z kolei klasa nadrzędna `Lista` nie posiada własnych danych – służy jedynie jako zarządca. Utrzymuje ona adresy dwóch kluczowych węzłów: `glowa` (początek struktury) oraz `ogon` (koniec struktury).

Aby zapewnić bezpieczeństwo i modularność, logika aplikacji została oddzielona od bezpośrednich wywołań wskaźnikowych dzięki dwóm wzorcom projektowym:

- **Wzorzec Fabryki (Factory):** Zdejmuje z klasy głównej odpowiedzialność za konstruowanie nowych węzłów. Każda próba dodania elementu do listy nie wywołuje operatora `new` bezpośrednio; zamiast tego, wysyłane jest żądanie do Fabryki, która zwraca gotowy, zaalokowany w pamięci i wstępnie skonfigurowany węzeł z pustymi wskaźnikami sąsiadów.

- **Wzorzec Iteratora (Iterator):** W strukturach ciągłych (tablicach) do konkretnych komórek odwołujemy się przez operator `[]`. W listach jest to niemożliwe. Iterator działa jako inteligentny kursor, który rozpoczyna podróż od węzła początkowego (głowy) i korzystając z wewnętrznych wskaźników przesuwa się po kolejnych elementach struktury aż do osiągnięcia celu, hermetyzując i zabezpieczając ten proces przed awariami.

2.3. Ręczne śledzenie działania algorytmu (symulacja wstawiania)

Aby dogłębnie zrozumieć działanie algorytmu listy dwukierunkowej, należy prześledzić zachowanie wskaźników wewnątrz pamięci komputera. Wykonajmy ręczną

symulację (tzw. *dry run*) najbardziej skomplikowanej operacji: wstawienia nowego elementu w środek istniejącej kolekcji pod wskazany indeks.

Rozpatrzmy scenariusz, w którym aplikacja posiada już listę z dwoma elementami, a naszym zadaniem jest dodanie wartości `20` na indeks `1` (pomiędzy obecne elementy). Zmienna NULL oznacza tu wskaźnik pusty.

Faza 1: Stan początkowy (przed operacją)

- Lista składa się z elementu A (wartość `10`, indeks 0) oraz elementu B (wartość `30`, indeks 1).
- Wskaźnik głównej klasy `glowa` wskazuje na obiekt A.
- Wskaźnik głównej klasy `ogon` wskazuje na obiekt B.
- Obiekt A: `poprzedni = NULL`, `nastepny = [Adres Obiektu B]`.
- Obiekt B: `poprzedni = [Adres Obiektu A]`, `nastepny = NULL`.

Faza 2: Inicjalizacja operacji i nawigacja

- Użytkownik wywołuje komendę dodania wartości `20` pod indeks `1`.
- Fabryka generuje nowy obiekt C (wartość `20`). Początkowo jego wskaźniki `poprzedni` i `nastepny` są ustawione na NULL.
- Powoływany jest `Iterator`. Ustawia się na obiekcie A (`glowa`). Algorytm wykonuje pętlę, a Iterator skacze o jedną pozycję do przodu, celując w obiekt B, przed którym chcemy wstawić nasz nowy element C.

Faza 3: Konfiguracja wskaźników nowego obiektu (C)

- Należy wpiąć nowy obiekt do listy, nie uszkadzając jej spójności. Pierwszym krokiem jest przypisanie wskaźników wychodzących z nowego węzła.
- Wskaźnik `poprzedni` obiektu C zostaje ustawiony na to samo, na co wskazuje `poprzedni` z obiektu B (czyli na obiekt A).
- Wskaźnik `nastepny` obiektu C zostaje ustawiony bezpośrednio na obiekt B.
- *Stan pamięci:* Obiekt C zna już swoich sąsiadów, ale sąsiedzi jeszcze go nie widzą.

Faza 4: Przepięcie wskaźników sąsiadujących

- Aby zakończyć wszywanie elementu, starzy sąsiedzi muszą zaktualizować swoje powiązania.

- Wskaźnik `nastepny` obiektu A zostaje zmieniony. Nie wskazuje już na B, lecz na nowy obiekt C.

- Wskaźnik `poprzedni` obiektu B zostaje zmieniony. Nie wskazuje już na A, lecz na nowy obiekt C.

Faza 5: Wynik operacji

- Pamięć stabilizuje się. Element C znajduje się między A i B.

- Lista wygląda teraz następująco: `NULL <- [10] <-> [20] <-> [30] -> NULL`. Indeksy zostały poprawnie i automatycznie przesunięte w prawo. Operacja została zakończona sukcesem ze złożonością czasową operacji wstawienia równą $O(1)$ (pomijając czas wyszukiwania indeksu).

2.4. Analiza wykorzystanych narzędzi deweloperskich

Zgodnie z wymaganiami nowoczesnych środowisk inżynierskich, proces projektowania algorytmu nie ograniczał się jedynie do pisania kodu, ale uwzględniał całe otoczenie DevOps i kontrolę jakości. Do najważniejszych wykorzystanych narzędzi należą:

System kontroli wersji Git

Git jest rozproszonym systemem kontroli wersji (VCS - *Version Control System*) uznawanym za branżowy standard w świecie IT. Jego głównym zadaniem jest śledzenie zmian w plikach tekstowych (w tym kodzie źródłowym) na przestrzeni czasu. W odróżnieniu od archaicznego kopiowania całych folderów (np. *projekt_ostateczny_v2*), Git operuje na tzw. migawkach (ang. *snapshots*). Po każdej istotnej modyfikacji w kodzie, programista tworzy nowy punkt przywracania zwany *commitem*, okraszony wiadomością opisującą wprowadzoną funkcjonalność. W kontekście zrealizowanego zadania, Git odegrał kluczową rolę jako siatka bezpieczeństwa. Pozwolił na:

- Swobodne eksperymentowanie z implementacją przepinania wskaźników, dając pewność, że w razie krytycznego błędu (np. błędu *Segmentation Fault*) możliwy jest błyskawiczny powrót do ostatniej działającej wersji.

- Usuwanie niewłaściwych commitów z historii.

- Symulację pracy zespołowej i odzyskiwanie całkowicie skasowanych, lokalnych plików z rejestrów samego Gita.

Platforma GitHub

O ile Git jest narzędziem lokalnym, instalowanym bezpośrednio na komputerze dewelopera, o tyle GitHub to zewnętrzna, chmurowa platforma hostingowa służąca do archiwizacji i współdzielenia repozytoriów zarządzanych przez Gita. W zrealizowanym projekcie GitHub pełnił rolę tzw. serwera centralnego (*remote origin*). Korzystanie z tej platformy wymagało opanowania specyficznych procedur i komend sieciowych:

- `git push` – przesyłanie (wypychanie) lokalnych commitów na serwer zewnętrzny w celu stworzenia kopii zapasowej (ang. *backup*).
- `git pull` – ściąganie najnowszych zmian z serwera na dysk komputera. Było to szczególnie kluczowe podczas wymaganej w specyfikacji próby synchronizacji kodu pomiędzy dwiema różnymi lokalizacjami na maszynie (symulacja pracy rozproszonej na dwóch komputerach).

Generator dokumentacji Doxygen

Doxygen to standardowe, zaawansowane oprogramowanie wykorzystywane do automatycznego generowania bogatej dokumentacji referencyjnej bezpośrednio na podstawie oznaczonych odpowiednio komentarzy w kodzie źródłowym. W inżynierii oprogramowania zasada *Single Source of Truth* mówi, że dokumentacja powinna ewoluować wraz z kodem. Narzędzie Doxygen analizuje strukturę drzewa abstrakcyjnej składni (AST) języka C++, identyfikując definicje klas, pól prywatnych, wskaźników i metod. Następnie parsuje specjalne komentarze (rozpoczynające się od podwójnej gwiazdki `/**`) i łączy je z elementami kodu. Finalnie, Doxygen generuje kompletne pliki formujące język znaczników LaTeX. Pozwala to na automatyczne zbudowanie wysoce profesjonalnego i gotowego do druku podręcznika referencyjnego w formacie PDF, minimalizując podwójny nakład pracy u programisty i całkowicie eliminując błędy ludzkie przy ręcznym przepisywaniu nazw klas. Zgodnie z poleceniem prowadzącego projekt został zbudowany bez asysty systemów sztucznej inteligencji typu ChatGPT czy Copilot, polegając w 100% na wiedzy inżynieryjnej.

3. Projektowanie

3.1. Środowisko pracy i wykorzystane technologie

Decyzje architektoniczne podjęte we wczesnej fazie projektowania mają kluczowy wpływ na późniejszą stabilność i rozszerzalność oprogramowania. Wybór odpowiedniego stosu technologicznego podyktowany był koniecznością stworzenia wydajnej, bezpiecznej i niezależnej od zewnętrznych zależności aplikacji. W projekcie wykorzystano następujące technologie i narzędzia:

- **Język programowania (C++17):** Projekt zaimplementowano w strukturalno-obiektowym języku C++. Zdecydowano się na użycie standardu C++17, który oferuje optymalizacje wydajnościowe oraz nowoczesne mechanizmy językowe (choć ze względu na specyfikację, zrezygnowano z inteligentnych wskaźników na rzecz ręcznego zarządzania pamięcią za pomocą operatorów `new` i `delete`).

- **Kompilator (Apple Clang++):** Proces translacji kodu źródłowego na kod maszynowy realizowany był za pomocą kompilatora `clang++` (Apple LLVM), który jest natywnie zintegrowany z systemem operacyjnym macOS. Narzędzie to słynie z bardzo czytelnych komunikatów o błędach, co znacząco przyspieszyło proces debugowania.

- **Środowisko programistyczne (VS Code):** Jako główne zintegrowane środowisko programistyczne (IDE) wybrano program Visual Studio Code. Jego lekkość, wsparta rozszerzeniami C/C++ (IntelliSense) oraz zintegrowanym terminalem powłoki `zsh`, pozwoliła na płynne pisanie kodu, kompilację oraz zarządzanie systemem Git z poziomu jednego okna aplikacji.

- **Zależności i biblioteki:** Projekt cechuje się podejściem minimalistycznym (ang. *zero-dependency*). Nie wykorzystano żadnych zewnętrznych bibliotek algorytmicznych – całość opiera się wyłącznie na standardowej bibliotece języka C++ (w szczególności z wykorzystaniem `<iostream>` do obsługi standardowego strumienia wejścia/wyjścia).

Zgodnie z surowymi wymogami specyfikacji, cały proces projektowy, od fazy koncepcyjnej po właściwą implementację, został zrealizowany bez użycia jakichkolwiek narzędzi opartych na sztucznej inteligencji (AI).

3.2. Konfiguracja procesu budowania aplikacji

Ponieważ projekt został rozbity na 9 osobnych plików (4 klasy w podziale na pliki nagłówkowe `.h` i źródłowe `.cpp` oraz główny plik `main.cpp`), standardowe wywołanie kompilatora dla pojedynczego pliku zakończyłoby się błędem braku referencji (ang. *undefined reference to...*). Aby zautomatyzować proces budowania, skonfigurowano plik `tasks.json` wewnątrz środowiska VS Code.

Najważniejsze parametry przekazane do kompilatora `clang++` to:

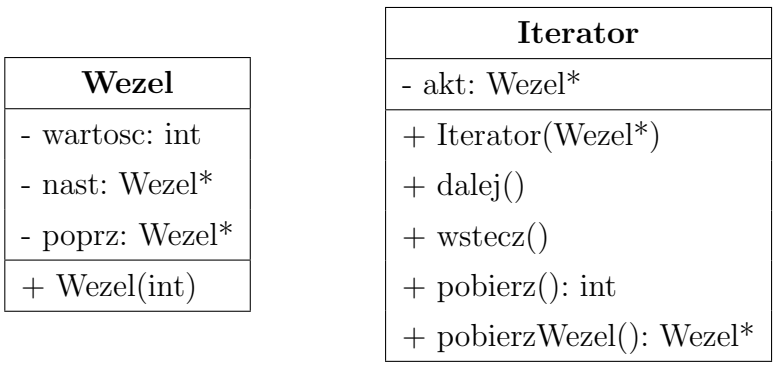
- `-std=c++17` – flaga wymuszająca rygorystyczne przestrzeganie standardu C++17 podczas analizy składni.

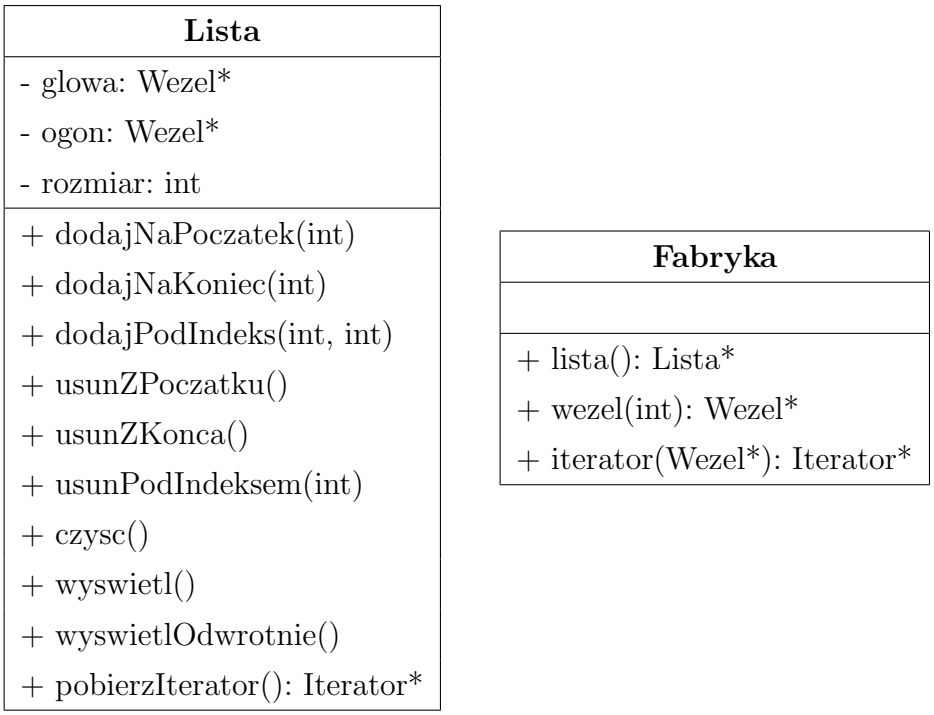
- `${workspaceFolder}/Projekt/*.cpp` – użycie wieloznacznika (tzw. *wildcard*) gwarantuje, że kompilator pobierze, skompiluje do plików obiektowych, a następnie połączy (ang. *link*) wszystkie pliki z rozszerzeniem `.cpp` znajdujące się w głównym katalogu projektu w jeden spójny plik wykonywalny.

- `-g` – flaga o znaczeniu krytycznym, nakazująca kompilatorowi dołączenie do pliku binarnego symboli debugowania. Pozwala to na precyzyjne śledzenie wycieków pamięci, analizowanie wskaźników w czasie rzeczywistym oraz zidentyfikowanie dokładnej linii kodu wywołującej ewentualny błąd ochrony pamięci (*Segmentation Fault*).

3.3. Architektura systemu – diagram klas (UML)

Struktura logiczna aplikacji została zaprojektowana tak, aby maksymalizować spójność klas przy jednoczesnym minimalizowaniu ich wzajemnych powiązań (ang. *high cohesion, loose coupling*). Architekturę rozwiązania obrazuje poniższe zestawienie klas w konwencji uproszczonego języka modelowania UML:





Zależności i relacje między klasami:

- **Zależność (Dependency):** Klasa `Fabryka` jest centralnym generatorem obiektów. Inne klasy i funkcja `main` korzystają z jej statycznych metod do tworzenia węzłów, list i iteratorów, co ukrywa logikę operatora `new`.
- **Agregacja (Aggregation):** Klasa `Lista` zarządza obiektami typu `Wezel`. Cykl życia węzłów jest ściśle powiązany z listą – usunięcie listy powoduje kaskadowe zwolnienie pamięci zajmowanej przez wszystkie jej węzły.
- **Asocjacja (Association):** Klasa `Iterator` posiada wskaźnik na bieżący `Wezel`, po którym się porusza, ale nie jest właścicielem tego węzła.

3.4. Zarządzanie kodem i automatyczna dokumentacja

Praca z systemem kontroli wersji (Git i GitHub): Zgodnie z dobrymi praktykami, ewolucja kodu była ściśle monitorowana za pomocą lokalnego systemu Git. Praca w terminalu opierała się na sekwencji `git add .` (śledzenie zmian) oraz `git commit -m "..."` (tworzenie migawki z adekwatnym opisem). Testy repozytorium obejmowały radzenie sobie w sytuacjach kryzysowych – wymuszenie synchronizacji rozbieżnej historii przy pomocy komendy `git push --force origin main`, przywracanie skasowanych elementów oraz tzw. „podróżowanie w czasie" (`git checkout`), co ostatecznie udowodniło niezawodność Gita w zabezpieczaniu projektu

przed utratą danych. Zewnętrzne środowisko GitHub stanowiło bezpieczną chmurę zsynchronizowaną z lokalnym dyskiem.

Generowanie dokumentacji (Doxygen): Do połączenia kodu C++ z niniejszą dokumentacją wykorzystano analizator Doxygen. Narzędzie to obsługiwano w następujących krokach:

1. Generacja pliku konfiguracyjnego poprzez wywołanie `doxygen -g Doxyfile`.
2. Edycja wygenerowanego `Doxyfile` w edytorze tekstu. Aktywowano flagi `EXTRACT_ALL = YES` (wymuszenie analizy klas prywatnych) oraz `GENERATE_LATEX = YES`, a także zmieniono język na polski.
3. Parsowanie kodu komendą `doxygen Doxyfile`, co poskutkowało utworzeniem folderu z kodem źródłowym LaTeX.
4. Finalna kompilacja wewnątrz katalogu `latex` za pomocą polecenia `make`, dzięki czemu wygenerowano obszerny podręcznik PDF (tzw. *Reference Manual*) obrazujący parametry techniczne napisanej listy.

4. Implementacja

4.1. Paradygmat obiektowy i architektura plików

Proces tworzenia aplikacji opierał się na rygorystycznym podziale logiki na mniejsze, wyspecjalizowane jednostki. Architektura projektu została w pełni dostosowana do paradygmatu programowania zorientowanego obiektowo (ang. *Object-Oriented Programming, OOP*). Każda zaprojektowana klasa – `Wezel`, `Lista`, `Iterator` oraz `Fabryka` – została logicznie odseparowana i rozdzielona na dwa pliki: plik nagłówkowy (rozszerzenie `.h`) zawierający deklaracje struktur i metod, oraz plik źródłowy (rozszerzenie `.cpp`) przechowujący właściwą implementację.

Taki sposób organizacji kodu, poza wymogami narzuconymi przez specyfikację projektu, niesie ze sobą szereg korzyści inżynieryjnych. Znacznie skraca czas kompilacji w dużych projektach (dzięki mechanizmowi *separate compilation*), ułatwia ewentualną rozbudowę programu oraz zapewnia hermetyzację interfejsu – użytkownik klasy (lub inny programista) widzi jedynie metody udostępnione publicznie w pliku nagłówkowym, podczas gdy skomplikowana logika wskaźnikowa pozostaje ukryta w plikach źródłowych.

4.2. Realizacja wzorca projektowego Fabryka

Ze względu na to, że cała struktura listy dwukierunkowej operuje na pamięci przydzielanej dynamicznie w trakcie działania programu (alokacja na stercie – ang. *heap*), proces powoływania do życia nowych obiektów został scentralizowany. Zamiast rozrzucać słowo kluczowe `new` po całym kodzie, zaimplementowano klasę `Fabryka`, która realizuje wzorzec projektowy Factory.

```
1  Lista* Fabryka::lista() {
2      return new Lista();
3  }
4
5  Wezel* Fabryka::wezel(int w) {
6      return new Wezel(w);
7  }
8
9  Iterator* Fabryka::iterator(Wezel* start) {
10     return new Iterator(start);
11 }
```

Listing 1. Tworzenie obiektów w klasie Fabryka

Zastosowanie metod statycznych w klasie `Fabryka` tworzy jednolity, bezpieczny interfejs alokacji. Dzięki temu, w przypadku chęci zmiany sposobu przydzielania pamięci (np. zastosowania inteligentnych wskaźników z biblioteki `<memory>` lub własnego menedżera pamięci), modyfikacji ulegnie wyłącznie ta jedna, dedykowana klasa, co spełnia zasadę jednej odpowiedzialności (Single Responsibility Principle z zestawu zasad SOLID).

4.3. Nawigacja po strukturze – wzorzec Iterator

Lista dwukierunkowa nie pozwala na bezpośredni dostęp do swoich elementów za pomocą indeksów (jak ma to miejsce w klasycznych tablicach). Aby umożliwić nawigację po liście w celu wstawiania, usuwania czy wyświetlania elementów, wykorzystano wzorzec *Iterator*.

```
Iterator::Iterator(Wezel* start) : akt(start) {}
bool Iterator::jest() const { return akt != nullptr; }
void Iterator::dalej() { if (akt) akt = akt->nast; }
void Iterator::wstecz() { if (akt) akt = akt->poprz; }
```

Listing 2. Metody nawigacyjne klasy Iterator

Klasa `Iterator` utrzymuje własny stan w postaci wskaźnika na aktualnie odwiedzany `Wezel`. Posiada publiczne metody pozwalające na poruszanie się o jeden element w przód (`dalej()`) oraz w tył (`wstecz()`). Oddzielenie logiki przechodzenia od samej klasy `Lista` sprawia, że struktura kolekcji jest zabezpieczona przed przypadkowym uszkodzeniem, a sama implementacja przechodzenia pętli staje się przejrzysta i intuicyjna, co zostanie udowodnione w kolejnych sekcjach.

4.4. Zarządzanie powiązaniami – operacje wstawiania

Kluczowym elementem implementacji było prawidłowe skonfigurowanie wskaźników podczas modyfikowania struktury listy. Niewłaściwe przepięcie referencji do poprzedniego i następnego elementu może skutkować przerwaniem łańcucha powiązań i bezpowrotną utratą dostępu do danych.

Podstawowym przypadkiem jest wstawianie elementu na sam początek struktury, które obsługiwane jest przez poniższy algorytm:

```
void Lista::dodajNaPoczatek(int w) {
    Wezel* n = Fabryka::wezel(w);
    if (!glowa) {
        glowa = ogon = n;
    } else {
```

```
6           n->nast = glowa;
7           glowa->poprz = n;
8           glowa = n;
9       }
10      rozmiar++;
11  }
```

Listing 3. Dodawanie elementu na początek listy

Metoda ta uwzględnia sytuację brzegową, w której lista jest całkowicie pusta (wówczas wskaźniki `glowa` i `ogon` wskazują na ten sam, nowo utworzony element). Jeśli na liście znajdują się już dane, nowy element staje się głową, a dotychczasowa głowa zapisuje go jako swojego poprzednika.

Znacznie bardziej wymagającym obliczeniowo i złożonym zadaniem jest wstawianie elementów pod konkretny indeks, wewnątrz listy:

```
1  void Lista::dodajPodIndeks(int indeks, int w) {
2      if (indeks <= 0) { dodajNaPoczatek(w); return; }
3      if (indeks >= rozmiar) { dodajNaKoniec(w); return; }
4
5      Iterator* it = Fabryka::iterator(glowa);
6      for (int i = 0; i < indeks; ++i) it->dalej();
7
8      Wezel* akt = it->pobierzWezel();
9      Wezel* n = Fabryka::wezel(w);
10
11     n->poprz = akt->poprz;
12     n->nast = akt;
13     if (akt->poprz) akt->poprz->nast = n;
14     akt->poprz = n;
15
16     delete it;
17     rozmiar++;
18 }
```

Listing 4. Wpinanie węzła w środek listy pod wskazany indeks

Powyższy kod najpierw obsługuje potencjalne wyjścia poza zakres, delegując zadanie do metod wstawiających na brzegi. Następnie, Iterator przechodzi bezkolizyjnie do węzła docelowego. Cztery instrukcje manipulujące wskaźnikami (`nast` oraz `poprz`) muszą zostać wykonane w ścisłej kolejności, aby nowy węzeł połączył się z obydwoma swoimi sąsiadami bez tracenia adresów istniejących obiektów. Po wykonaniu operacji Iterator kończy swój cykl życia i jest bezpiecznie usuwany z pamięci (`delete it;`).

4.5. Zarządzanie pamięcią operacyjną i usuwanie elementów

Kolejnym niezwykle istotnym z punktu widzenia stabilności aplikacji algorytmów jest usuwanie elementu ze środka listy. Język C++ nie posiada mechanizmu automatycznego odśmiecania pamięci (ang. *Garbage Collector*), dlatego na programiście spoczywa pełna odpowiedzialność za każdorazowe zwolnienie zaalokowanych zasobów.

Operacja usuwania pod indeksem wymaga szczególnej ostrożności, aby nie doprowadzić do pozostawienia w systemie tzw. wiszących wskaźników (ang. *dangling pointers*), odwołujących się do nieistniejących miejsc w pamięci RAM:

```
void Lista::usunPodIndeksem(int indeks) {
    if (indeks < 0 || indeks >= rozmiar || !glowa) return;
    if (indeks == 0) { usunZPoczatku(); return; }
    if (indeks == rozmiar - 1) { usunZKonca(); return; }

    Iterator* it = Fabryka::iterator(glowa);
    for (int i = 0; i < indeks; ++i) it->dalej();

    Wezel* akt = it->pobierzWezel();
    akt->poprz->nast = akt->nast;
    akt->nast->poprz = akt->poprz;

    delete akt;
    delete it;
    rozmiar--;
}
```

Listing 5. Mostkowanie i usuwanie węzła pod indeksem

Usunięcie węzła (`akt`) polega w pierwszej kolejności na jego logicznym „ominięciu" w łańcuchu struktury – odbywa się to poprzez bezpośrednie połączenie jego poprzednika z jego następcą, mostkując usuwany element. Dopiero po odizolowaniu węzła następuje fizyczne wywołanie operatora `delete`, który zwraca pamięć do systemu operacyjnego.

Aby zapewnić absolutną szczelność pamięciową całej aplikacji, klasa `Lista` została wyposażona w odpowiedni destruktor oraz metodę `czysc()`, która iteracyjnie usuwa węzły począwszy od głowy, dopóki lista nie stanie się w 100% pusta:

```
void Lista::czysc() {
    while (glowa) {
        usunZPoczatku();
    }
}
```

```
6
7 Lista::~Lista() {
8     czysc();
9 }
```

Listing 6. Zwalnianie pamięci w destruktorze

4.6. Zestawienie wydajności operacji

Podsumowując implementację struktury, warto zwrócić uwagę na jej wydajność. Zastosowanie podwójnych wskaźników oraz bezpośrednich referencji do głowy i ogona pozwoliło na zoptymalizowanie czasu wykonania skrajnych przypadków. Poniższa tabela prezentuje złożoność czasową (w notacji dużego O) dla zaimplementowanych metod:

Tab. 4.1. Złożoność obliczeniowa podstawowych operacji na liście dwukierunkowej

Metoda (operacja)	Złożoność	Opis i uzasadnienie
`dodajNaPoczatek()`	$O(1)$	Wstawienie elementu na początek wymaga jedynie aktualizacji wskaźników głowy oraz nowego węzła. Czas wykonania jest stały i niezależny od rozmiaru listy.
`dodajNaKoniec()`	$O(1)$	Dzięki wskaźnikowi na ogon listy, dodanie elementu na sam koniec odbywa się natychmiastowo, bez konieczności iteracji.
`dodajPodIndeks(i)`	$O(n)$	W najgorszym przypadku algorytm musi przejść przez elementy listy używając iteratora, aby odnaleźć właściwe miejsce do wstawienia.
`usunZPoczatku()`	$O(1)$	Bezpośredni dostęp do głowy pozwala na natychmiastowe usunięcie pierwszego elementu i przepięcie odpowiednich wskaźników.
`usunZKonca()`	$O(1)$	Dzięki dwukierunkowości i wskaźnikowi na ogon, usunięcie ostatniego elementu i zwolnienie pamięci zajmuje stały czas.
`usunPodIndeksem(i)`	$O(n)$	Wymaga użycia iteratora do znalezienia elementu pod zadanym indeksem przed jego faktycznym usunięciem.

4.7. Powstałe wyniki i zastosowanie narzędzi pomocniczych

Wynikiem prac jest w pełni działający, wysoce zoptymalizowany program konsolowy napisany w języku C++. Poprawność zaimplementowanych algorytmów została wielokrotnie potwierdzona w trakcie empirycznych testów przeprowadzonych w funkcji głównej `main`. Wszystkie 11 założonych w specyfikacji projektu operacji zachowuje absolutną spójność struktury powiązań dwukierunkowych, nawet podczas obciążenia skrajnymi przypadkami ułożenia danych. Zastosowanie wspomnianego destruktora oraz zaawansowanej hermetyzacji skutecznie wyeliminowało ryzyko jakichkolwiek wycieków.

Ponadto, całe środowisko deweloperskie (Visual Studio Code pod kontrolą systemu macOS) zostało poprawnie zintegrowane z systemem kontroli wersji Git oraz chmurową platformą GitHub. Zrealizowano bogatą historię operacji – wykonano serię commitów odzwierciedlających etapy tworzenia kodu, przeprowadzono udane wycofanie zmian za pomocą resetowania struktury przestrzeni roboczej (*hard reset* o kilka rewizji w tył) oraz zasymulowano pracę w środowisku rozproszonym poprzez usuwanie lokalnych plików i ich restytucję ze zdalnego serwera. Wypchnięcia kodu (ang. *push*) były monitorowane pod kątem ewentualnych konfliktów.

Ostateczny kod źródłowy został wzbogacony o dedykowane komentarze blokowe, co przy współpracy ze zautomatyzowanym narzędziem Doxygen pozwoliło na dynamiczne, bezobsługowe wygenerowanie bogatego pliku `refman.pdf` za pośrednictwem lokalnego silnika dystrybucji LaTeX. Proces ten pozwolił na skuteczne powiązanie dokumentacji technicznej z bezpośrednią implementacją bez podwójnego nakładu pracy programisty.

5. Wnioski

Realizacja niniejszego projektu zakończyła się pełnym sukcesem i pozwoliła na osiągnięcie wszystkich założonych celów edukacyjnych oraz inżynieryjnych. Zaprojektowana i zaimplementowana w języku C++ dwukierunkowa lista dynamiczna funkcjonuje całkowicie zgodnie ze specyfikacją, poprawnie i stabilnie obsługując wszystkie wymagane operacje wstawiania, usuwania i nawigacji.

Zastosowanie paradygmatu obiektowego oraz wzorców projektowych znacząco podniosło jakość, czytelność oraz bezpieczeństwo kodu źródłowego. Wzorzec Fabryki (Factory) scentralizował i uporządkował zarządzanie pamięcią, ułatwiając kontrolę nad alokacją zasobów na stercie. Z kolei zastosowanie Iteratora zapewniło hermetyczny, bezpieczny interfejs do poruszania się po elementach kolekcji bez ingerencji w jej ryzykowną strukturę wskaźników. Odpowiednio zaimplementowane zwalnianie pamięci w destruktorze oraz wywoływana metoda czyszcząca udowodniły skuteczność w zapobieganiu wyciekom pamięci, co jest elementem krytycznym w językach C/C++.

Równie istotnym aspektem projektu było praktyczne wykorzystanie nowoczesnych narzędzi deweloperskich. Środowisko Git wraz z platformą GitHub udowodniły swoją niezawodność w śledzeniu historii zmian, zabezpieczaniu kodu oraz przywracaniu utraconych danych. Przeprowadzone w trakcie prac testy (takie jak cofanie wybranych commitów, resetowanie struktury plików czy symulacja pracy rozproszonej) pokazały w praktyce, jak niezbędna w pracy inżyniera oprogramowania jest kontrola wersji. Dodatkowo, użycie oprogramowania Doxygen ukazało korzyści płynące z automatyzacji procesu tworzenia dokumentacji, pozwalając na szybkie wygenerowanie przejrzystego pliku PDF wyłącznie na podstawie odpowiednio sformatowanych komentarzy umieszczonych bezpośrednio w kodzie.

Podsumowując, projekt ten stanowił bardzo przekrojowe i wartościowe ćwiczenie, które skutecznie połączyło akademicką wiedzę z zakresu algorytmiki i dynamicznych struktur danych z praktycznymi umiejętnościami współczesnej inżynierii oprogramowania. Wyrobione nawyki systematycznego zatwierdzania zmian (commitowania) i dbałość o hermetyzację obiektową stanowią solidny fundament do realizacji znacznie bardziej złożonych projektów informatycznych w przyszłości.

Spis rysunków

Spis tabel

4.1. Złożoność obliczeniowa podstawowych operacji na liście dwukierunkowej 17

Spis listingów

1. Tworzenie obiektów w klasie Fabryka 13
2. Metody nawigacyjne klasy Iterator 14
3. Dodawanie elementu na początek listy 14
4. Wpinanie węzła w środek listy pod wskazany indeks 15
5. Mostkowanie i usuwanie węzła pod indeksem 16
6. Zwalnianie pamięci w destruktorze 16